

C PREPROCESSOR

CEN116 Algorithms and Programming

Introduction

- The C preprocessor executes before a program is compiled.
- Some actions it performs are the inclusion of other files in the file being compiled, definition of symbolic constants and macros, conditional compilation of program code and conditional execution of preprocessor directives.
- Preprocessor directives begin with `#` and only whitespace characters and comments may appear before a preprocessor directive on a line.

#include Preprocessor Directive

- The #include directive causes a copy of a specified file to be included in place of the directive.
- The two forms of the #include directive are:
 - `#include <filename>`
 - `#include "filename"`
- The difference between these is the location the preprocessor begins searches for the file to be included.
- If the file name is enclosed in quotes, the preprocessor starts searches in the same directory as the file being compiled for the file to be included (and may search other locations, too).

#include Preprocessor Directive

- This method is normally used to include programmer-defined headers.
- If the file name is enclosed in angle brackets (< and >)—used for standard library headers—the search is performed in an implementation-dependent manner, normally through predesignated compiler and system directories.

#include Preprocessor Directive

- The #include directive is used to include standard library headers such as `stdio.h` and `stdlib.h` and with programs consisting of multiple source files that are to be compiled together.
- A header containing declarations common to the separate program files is often created and included in the file.
- Examples of such declarations are structure and union declarations, enumerations and function prototypes.

#define Preprocessor Directive: Symbolic Constants

- The #define directive creates symbolic constants—constants represented as symbols—and macros—operations defined as symbols.
- The #define directive format is
#define identifier replacement-text
- When this line appears in a file, all subsequent occurrences of identifier that do not appear in string literals will be replaced by replacement-text automatically before the program is compiled.

#define Preprocessor Directive: Symbolic Constants

- For example, `#define PI 3.14159` replaces all subsequent occurrences of the symbolic constant `PI` with the numeric constant `3.14159`.
- Symbolic constants enable you to create a name for a constant and use the name throughout the program.
- If the constant needs to be modified throughout the program, it can be modified once in the `#define` directive.
- When the program is recompiled, all occurrences of the constant in the program will be modified accordingly.

#define Preprocessor Directive: Symbolic Constants

- Everything to the right of the symbolic constant name replaces the symbolic constant.
- For example, `#define PI =3.14159` causes the preprocessor to replace every occurrence of the identifier `PI` with `=3.14159`.
- This is the cause of many subtle logic and syntax errors.
- For this reason, you may prefer to use `const` variable declarations, such as `const double PI = 3.14159;` in preference to the preceding `#define`.
- Redefining a symbolic constant with a new value is also an error.

#define Preprocessor Directive: Macros

- A macro is an identifier defined in a `#define` preprocessor directive.
- As with symbolic constants, the macro-identifier is replaced in the program with the replacement-text before the program is compiled.
- Macros may be defined with or without arguments.
- A macro without arguments is processed like a symbolic constant.
- In a macro with arguments, the arguments are substituted in the replacement text, then the macro is expanded—i.e., the replacement-text replaces the identifier and argument list in the program.

#define Preprocessor Directive: Macros

- Consider the following macro definition with one argument for the area of a circle:

```
#define CIRCLE_AREA(x) ((PI) * (x) * (x))
```

- Wherever CIRCLE_AREA(y) appears in the file, the value of y is substituted for x in the replacement-text, the symbolic constant PI is replaced by its value (defined previously) and the macro is expanded in the program

#define Preprocessor Directive: Macros

- For example, the statement *area = CIRCLE_AREA(4);* is expanded to *area = ((3.14159) * (4) * (4));* then, at compile time, the value of the expression is evaluated and assigned to variable *area*.
- The parentheses around each *x* in the replacement text force the proper order of evaluation when the macro argument is an expression.

#define Preprocessor Directive: Macros

- For example, the statement

area = CIRCLE_AREA(c + 2);

is expanded to

*area = ((3.14159) * (c + 2) * (c + 2));*

which evaluates correctly because the parentheses force the proper order of evaluation.

#define Preprocessor Directive: Macros

- If the parentheses in the macro definition are omitted, the macro expansion is
 - $area = 3.14159 * c + 2 * c + 2;$
which evaluates incorrectly as
 - $area = (3.14159 * c) + (2 * c) + 2;$
because of the rules of operator precedence

#define Preprocessor Directive: Macros

- The following is a macro definition with two arguments for the area of a rectangle:
 - *#define RECTANGLE_AREA(x, y) ((x) * (y))*
- Wherever RECTANGLE_AREA(x, y) appears in the program, the values of x and y are substituted in the macro replacement text and the macro is expanded in place of the macro name.
- For example, the statement
 - *rectArea = RECTANGLE_AREA(a + 4, b + 7);*
 - is expanded to
 - *rectArea = ((a + 4) * (b + 7));*

#define Preprocessor Directive: Macros

- The value of the expression is evaluated at runtime and assigned to variable rectArea.
- The replacement text for a macro or symbolic constant is normally any text on the line after the identifier in the #define directive.
- If the replacement text for a macro or symbolic constant is longer than the remainder of the line, a backslash (\) must be placed at the end of the line, indicating that the replacement text continues on the next line.

#define Preprocessor Directive: Macros

- Symbolic constants and macros can be discarded by using the `#undef` preprocessor directive.
- Directive `#undef` “undefines” a symbolic constant or macro name.
- The scope of a symbolic constant or macro is from its definition until it’s undefined with `#undef`, or until the end of the file.
- Once undefined, a name can be redefined with `#define`.

Conditional Compilation

- Conditional compilation enables you to control the execution of preprocessor directives and the compilation of program code.
- Each conditional preprocessor directive evaluates a constant integer expression.
- The conditional preprocessor construct is much like the if selection statement.

Conditional Compilation

- Consider the following preprocessor code:
 - *#if !defined(MY_CONSTANT)*
#define MY_CONSTANT 0
#endif
- Which determines whether MY_CONSTANT is defined—that is, whether MY_CONSTANT has already appeared in an earlier #define directive.
- The expression defined(MY_CONSTANT) evaluates to 1 if MY_CONSTANT is defined and 0 otherwise.
- If the result is 0, !defined(MY_CONSTANT) evaluates to 1 and MY_CONSTANT is defined.
- Otherwise, the #define directive is skipped.

Conditional Compilation

- Every `#if` construct ends with `#endif`.
- Directives `#ifdef` and `#ifndef` are shorthand for `#if defined(name)` and `#if !defined(name)`.

References

- C How to Program, 8ed, by Paul Deitel and Harvey Deitel, Pearson, 2016.